

DIGILIANS Initiative

Capstone Project

Full Stack Development Track

Sa2yanti



Car Maintenance Services Platform

Prepared By:

**Rabea Shaban**

Individual Project

Submitted in Partial Fulfillment of the Requirements for the  
Capstone Project – Full Stack Development Track

DIGILIANS Initiative

2026

وزارة الاتصالات  
وتكنولوجيا المعلومات



| Page | Content                       |
|------|-------------------------------|
| 1    | Cover Page                    |
| 2    | Abstract                      |
| 3-4  | Introduction                  |
| 5-6  | Problem Definition            |
| 7    | Project Objectives            |
| 8    | System Architecture           |
| 9    | Frontend Architecture         |
| 10   | Backend Architecture          |
| 11   | Database Design               |
| 12   | Implementation & Dependencies |
| 13   | Achievements & Progress       |
| 14   | Challenges & Future Work      |
| 15   | References                    |

# الملخص (Abstract)

**صيانتي (Sa2yanti)** هي منصة إلكترونية تعمل عبر الويب وتهدف إلى ربط أصحاب السيارات بفنيي ومراكز الصيانة من خلال نظام رقمي موحد. تساعد المنصة المستخدمين على طلب خدمات الصيانة بسهولة، ومتابعة حالة الطلبات، وتحسين عملية التواصل بين العملاء ومقدمي الخدمات.

تم تطوير المنصة باستخدام مجموعة من التقنيات الحديثة في مجال تطوير تطبيقات الويب، حيث تم استخدام **React.js** و **TypeScript** لتطوير واجهة المستخدم، بينما تم استخدام **Node.js** و **Express.js** لتطوير الخادم ومعالجة العمليات الخلفية، بالإضافة إلى **MongoDB Atlas** لتخزين وإدارة البيانات.

ولضمان حماية بيانات المستخدمين وتأمين عمليات تسجيل الدخول، تم تطبيق نظام مصادقة يعتمد على **JWT (JSON Web Token)** و **HTTP Only Cookies**، إلى جانب نظام لإدارة الصلاحيات يعتمد على أدوار المستخدمين (**Role-Based Access Control - RBAC**).

يركز الإصدار الحالي من المنصة على الوظائف الأساسية، مثل إنشاء الحسابات، وتسجيل الدخول بشكل آمن، وإدارة طلبات الصيانة. كما يمكن للمستخدم إنشاء طلب صيانة ومتابعة حالته من خلال واجهة سهلة الاستخدام ومنظمة.

تم تصميم النظام بطريقة تسمح بالتوسع والتطوير مستقبلاً، مما يتيح إضافة مزايا جديدة مثل لوحات التحكم الخاصة بالفنيين ومراكز الصيانة، ونظام الإشعارات، وتتبع حالة الخدمات بشكل مباشر، بالإضافة إلى الخدمات المعتمدة على الموقع الجغرافي.

وتهدف منصة **صيانتي** إلى تحسين تجربة صيانة السيارات من خلال توفير حل رقمي آمن وفعال يساعد على تنظيم عمليات الصيانة وتسهيل التواصل بين العملاء ومقدمي الخدمات، بما يحقق تجربة أفضل لجميع الأطراف.

## 1. المقدمة - (Introduction)

مع الزيادة المستمرة في أعداد السيارات، أصبحت خدمات الصيانة من الخدمات الأساسية التي يحتاج إليها أصحاب المركبات بشكل مستمر. ومع ذلك، لا يزال الكثير من المستخدمين يواجهون صعوبة في الوصول إلى فنيين موثوقين أو مراكز صيانة مناسبة، بالإضافة إلى التحديات المتعلقة بمتابعة طلبات الصيانة ومعرفة حالة الخدمة بشكل واضح.

في الوقت الحالي، تعتمد معظم عمليات الصيانة على المكالمات الهاتفية أو تطبيقات التواصل الاجتماعي، وهي طرق قد تكون غير منظمة وتستهلك وقتاً وجهداً من العملاء ومقدمي الخدمات. كما أن هذه الأساليب لا توفر وسيلة واضحة لمتابعة الطلبات أو إدارة البيانات بشكل فعال.

لذلك تم تطوير منصة **صيانتي (Sa2yanti)** ، وهي منصة إلكترونية تعمل عبر الويب وتهدف إلى ربط أصحاب السيارات بفنيي ومراكز الصيانة من خلال نظام رقمي موحد. تتيح المنصة للمستخدمين إنشاء طلبات الصيانة ومتابعتها بسهولة، كما تساعد مقدمي الخدمات على إدارة الطلبات بشكل أكثر تنظيماً وكفاءة.

تم تطوير المشروع باستخدام مجموعة من التقنيات الحديثة في مجال تطوير تطبيقات الويب، حيث تم استخدام **React.js** و **TypeScript** لتطوير واجهة المستخدم، بينما تم استخدام **Node.js** و **Express.js** لتطوير الخادم والتعامل مع العمليات الخلفية، بالإضافة إلى **MongoDB Atlas** لتخزين وإدارة البيانات.

ولضمان حماية بيانات المستخدمين وتأمين عمليات تسجيل الدخول، تم تطبيق نظام مصادقة يعتمد على **JWT (JSON Web Token)** و **HTTP Only Cookies**، مما يوفر مستوى أعلى من الأمان أثناء استخدام المنصة.

وتهدف منصة صيانتني إلى تحسين تجربة صيانة السيارات من خلال توفير بيئة رقمية آمنة وسهلة الاستخدام، تساعد على تسهيل التواصل بين العملاء ومقدمي الخدمات، وتنظيم عمليات الصيانة بشكل أكثر كفاءة وقابلية للتطوير مستقبلاً.

## 2. تعريف المشكلة (Problem Definition)

مع زيادة عدد السيارات، بقت خدمات الصيانة جزء مهم في حياة الناس. لكن لسه في مشاكل كتير بتواجه أصحاب السيارات لما يحتاجوا فني أو مركز صيانة مناسب، زي صعوبة الوصول لفني موثوق، أو متابعة حالة طلب الصيانة، أو معرفة آخر التحديثات الخاصة بالخدمة.

في أغلب الأحيان بيتم التواصل بين العميل والفني عن طريق المكالمات الهاتفية أو تطبيقات المراسلة، وده بيخلي إدارة الطلبات ومتابعتها أمر صعب وغير منظم. كمان ممكن يحصل تأخير أو سوء فهم بين الطرفين بسبب عدم وجود نظام واضح لإدارة الطلبات.

من ناحية ثانية، مراكز الصيانة والفنيين بيواجهوا صعوبة في إدارة عدد كبير من الطلبات بشكل يدوي، خاصة مع زيادة عدد العملاء. بالإضافة إلى ذلك، بعض الأنظمة المتاحة لا توفر مستوى أمان كافٍ لحماية بيانات المستخدمين أو إدارة الصلاحيات بشكل صحيح.

عشان كده ظهرت الحاجة إلى منصة رقمية تجمع بين أصحاب السيارات ومقدمي خدمات الصيانة في مكان واحد، وتساعد على تنظيم الطلبات وتحسين التواصل بين الطرفين بشكل أسهل وأسرع.

منصة صيانتني (Sa2yanti) اتعملت لحل المشاكل دي من خلال نظام إلكتروني يتيح للمستخدم إنشاء طلب صيانة ومتابعته بسهولة، وفي نفس الوقت يساعد الفنيين ومراكز الصيانة على إدارة الطلبات وتحديث حالتها بشكل منظم وآمن.

## 3. أهداف المشروع (Project Objectives)

يهدف مشروع صيانتني (Sa2yanti) إلى تطوير منصة إلكترونية تعمل عبر الويب لتسهيل عملية طلب وإدارة خدمات صيانة السيارات. وتسعى المنصة إلى توفير بيئة رقمية آمنة ومنظمة تربط بين أصحاب السيارات ومقدمي خدمات الصيانة، مما يساعد على تحسين تجربة المستخدم وتسهيل إدارة الطلبات بشكل أكثر كفاءة.

يركز المشروع على تحقيق مجموعة من الأهداف الرئيسية، وهي:

- توفير منصة مركزية لإدارة واستقبال طلبات الصيانة.
- تسهيل الوصول إلى خدمات الصيانة بشكل أسرع وأكثر تنظيماً.
- تطبيق آليات آمنة للمصادقة وإدارة الصلاحيات.
- تمكين المستخدمين من إنشاء وإدارة طلبات الصيانة بسهولة.
- تحسين تجربة المستخدم من خلال واجهة استخدام مرنة ومتجاوبة مع مختلف الأجهزة.
- تطوير نظام يعتمد على بنية Full Stack قابلة للتوسع والتطوير.
- استخدام أحدث التقنيات والممارسات في تطوير البرمجيات.
- توفير أساس قوي يسمح بإضافة مزايا جديدة في المستقبل.

من المتوقع أن يساهم المشروع بعد اكتماله في تحقيق عدد من النتائج المهمة، منها:

- تحسين التواصل بين أصحاب السيارات والفنيين ومقدمي الخدمات.
- تسهيل إدارة ومتابعة طلبات الصيانة.
- توفير بيئة آمنة وموثوقة لإدارة الخدمات والبيانات.
- زيادة الشفافية في جميع مراحل عملية الصيانة.
- إنشاء بنية تقنية قابلة للتطوير لدعم التوسعات المستقبلية.

ومن خلال تحقيق هذه الأهداف، تسعى منصة صيانتني إلى تقديم حل رقمي عملي يساعد على تحسين تجربة صيانة السيارات، ويوفر وسيلة أكثر سهولة وكفاءة للتعامل بين العملاء ومقدمي خدمات الصيانة.

## 4. هيكلية النظام(System Architecture)

تعتمد منصة صيانتني (Sa2yanti) على هيكلية مكونة من ثلاث طبقات رئيسية، وهي: واجهة المستخدم (Frontend) ، والخادم (Backend) ، وقاعدة البيانات (Database). يساعد هذا التصميم على تنظيم مكونات النظام بشكل واضح، مما يسهل عملية التطوير والصيانة والتوسع مستقبلاً، بالإضافة إلى تعزيز مستوى الأمان وتحسين أداء النظام.

### نظرة عامة على الهيكلية(Architecture Overview)

المستخدم  
↓  
واجهة المستخدم (React.js)  
↓  
REST API (Node.js & Express.js)  
↓  
قاعدة البيانات (MongoDB Atlas)

### أولاً: طبقة واجهة المستخدم(Frontend Layer)

تم تطوير واجهة المستخدم باستخدام **React.js** و **TypeScript**، وهي المسؤولة عن عرض البيانات والتفاعل مع المستخدم وإدارة صفحات النظام المختلفة.

### التقنيات المستخدمة:

- React.js
- TypeScript
- Tailwind CSS
- React Router DOM
- Axios
- Context API

## المسؤوليات الرئيسية:

- عرض واجهات المستخدم.
- إدارة التنقل بين الصفحات.
- التعامل مع النماذج وإدخال البيانات.
- التحقق من صحة البيانات قبل إرسالها.
- التواصل مع واجهات البرمجة (APIs) الخاصة بالخادم.

## ثانيًا: طبقة الخادم (Backend Layer)

تم تطوير الخادم باستخدام **Node.js** و **Express.js**، وهو المسؤول عن تنفيذ منطق العمل (Business Logic) وإدارة العمليات المختلفة داخل النظام.

## التقنيات المستخدمة:

- Node.js
- Express.js
- JWT
- HTTP Only Cookies
- Joi Validation
- bcryptjs

## المسؤوليات الرئيسية:

- إدارة عمليات تسجيل الدخول وإنشاء الحسابات.
- التحقق من صلاحيات المستخدمين.
- التحقق من صحة البيانات المستلمة.
- إدارة طلبات الصيانة والعمليات المرتبطة بها.
- إنشاء وتوفير واجهات البرمجة (REST APIs).
- التواصل مع قاعدة البيانات.

## ثالثًا: طبقة قاعدة البيانات (Database Layer)

تعتمد المنصة على **MongoDB Atlas** كقاعدة بيانات رئيسية لتخزين وإدارة بيانات النظام.

## البيانات المخزنة:

- بيانات المستخدمين.
- طلبات الصيانة.
- بيانات الفنيين المرتبطين بالطلبات.
- حالات الطلبات وتحديثاتها.

## التقنيات المستخدمة:

- MongoDB Atlas
- Mongoose ODM

تتم عملية تسجيل الدخول والتحقق من هوية المستخدم وفق الخطوات التالية:

1. يقوم المستخدم بتسجيل الدخول باستخدام البريد الإلكتروني وكلمة المرور.
2. يتحقق الخادم من صحة البيانات المدخلة.
3. يتم إنشاء رمز مصادقة (JWT) للمستخدم.
4. يتم حفظ الرمز داخل HTTP Only Cookie لزيادة مستوى الأمان.
5. عند الوصول إلى الصفحات المحمية، يتم التحقق من الرمز من خلال Middleware مخصص.
6. في حالة نجاح التحقق، يتم السماح للمستخدم بالوصول إلى الموارد المصرح بها.

### مميزات الهيكلية المستخدمة

توفر هذه الهيكلية مجموعة من المميزات المهمة، منها:

- فصل واضح بين مكونات النظام ومسؤوليات كل جزء.
- توفير مستوى عالٍ من الأمان للمستخدمين والبيانات.
- سهولة الربط والتواصل بين الواجهة الأمامية والخادم.
- سهولة صيانة النظام وإضافة تعديلات أو مزايا جديدة.
- دعم التوسع المستقبلي وإضافة خصائص متقدمة دون الحاجة إلى إعادة بناء النظام بالكامل.

## 5. هيكلية الواجهة الأمامية (Frontend Architecture)

تمثل الواجهة الأمامية (Frontend) الجزء المسؤول عن تفاعل المستخدم مع منصة صيانتي (Sa2yanti)، حيث توفر واجهة سهلة الاستخدام ومتجاوبة تساعد أصحاب السيارات ومقدمي خدمات الصيانة على استخدام النظام وإدارة الطلبات بكفاءة.

تم تطوير الواجهة الأمامية باستخدام **React.js** و **TypeScript** مع الاعتماد على أسلوب **Component-Based Architecture**، والذي يساهم في تنظيم الكود وإعادة استخدام المكونات بسهولة، مما يسهل عمليات التطوير والصيانة مستقبلاً.

### التقنيات المستخدمة (Technologies Used)

تم بناء الواجهة الأمامية باستخدام التقنيات التالية:

- React.js
- TypeScript
- Tailwind CSS
- React Router DOM
- React Hook Form
- Zod
- Axios
- Context API

تم تنظيم المشروع بطريقة مرنة ومنظمة لتسهيل إدارة الملفات وتطوير المزايا الجديدة، ويتكون الهيكل الرئيسي من:

- Components
- Pages
- Context
- Hooks
- Routes
- Services

يساعد هذا التنظيم على فصل المسؤوليات بين أجزاء المشروع المختلفة، مما يجعل الكود أكثر وضوحًا وأسهل في التطوير والصيانة.

## إدارة المصادقة (Authentication Flow)

تم تنفيذ نظام المصادقة باستخدام **Context API** لإدارة حالة المستخدم داخل التطبيق، مع الاعتماد على **HTTP Only Cookies** لتخزين رمز المصادقة بشكل آمن.

خطوات تسجيل الدخول:

1. يقوم المستخدم بإدخال بيانات تسجيل الدخول.
2. يتم إرسال البيانات إلى الخادم للتحقق من صحتها.
3. في حالة نجاح عملية التحقق، يقوم الخادم بإنشاء رمز مصادقة (JWT).
4. يتم تخزين الرمز داخل **HTTP Only Cookie**.
5. يتم جلب بيانات المستخدم وحفظها داخل **Context API**.
6. يصبح بإمكان المستخدم الوصول إلى الصفحات المحمية وفقًا لصلاحياته.

## الصفحات المنفذة (Implemented Pages)

يتضمن الإصدار الحالي من المنصة مجموعة من الصفحات الأساسية، منها:

- الصفحة الرئيسية (Landing Page)
- صفحة تسجيل الدخول (Login Page)
- صفحة إنشاء حساب جديد (Register Page)
- الصفحات المحمية (Protected Pages)

## مميزات هيكلية الواجهة الأمامية

توفر هذه الهيكلية العديد من المزايا، من أهمها:

- واجهة استخدام متجاوبة تعمل على مختلف الأجهزة والشاشات.
- إمكانية إعادة استخدام المكونات في أكثر من مكان داخل النظام.
- سهولة التكامل مع واجهات البرمجة (APIs).
- دعم نظام مصادقة آمن ومتوافق مع أفضل الممارسات.
- سهولة إضافة مزايا جديدة وتطوير النظام مستقبلاً.



وتوفر هذه الهيكلية أساسًا قويًا لتطوير مزايا إضافية مستقبلاً، مثل متابعة حالة طلبات الصيانة، ولوحات التحكم الخاصة بالفنيين ومراكز الصيانة، وإضافة خدمات جديدة داخل المنصة.

## 6. هيكلية الخادم (Backend Architecture)

يمثل الخادم (Backend) الجزء المسؤول عن تنفيذ منطق العمل داخل منصة صيانتنا (Sa2yanti)، حيث يتولى إدارة عمليات تسجيل الدخول، والتحقق من صلاحيات المستخدمين، ومعالجة طلبات الصيانة، والتواصل مع قاعدة البيانات.

تم تطوير الخادم باستخدام **Node.js** و **Express.js** مع الاعتماد على أسلوب **RESTful API Architecture**، مما يوفر طريقة منظمة وأمنة للتواصل بين الواجهة الأمامية وقاعدة البيانات.

### التقنيات المستخدمة (Technologies Used)

تم بناء الخادم باستخدام مجموعة من التقنيات الحديثة، وهي:

- Node.js
- Express.js
- MongoDB Atlas
- Mongoose
- JWT
- HTTP Only Cookies
- Joi Validation
- bcryptjs

### هيكل المشروع (Backend Structure)

تم تصميم المشروع بطريقة منظمة تعتمد على فصل المسؤوليات بين المكونات المختلفة، ويتكون من:

- Models
- Controllers
- Routes
- Middleware
- Validators
- Config

يساعد هذا التنظيم على تحسين جودة الكود، وتسهيل عمليات الصيانة، وإضافة مزايا جديدة دون التأثير على الأجزاء الأخرى من النظام.

### المصادقة وإدارة الصلاحيات (Authentication and Authorization)

تم تطبيق نظام مصادقة آمن باستخدام **JWT (JSON Web Token)** و **HTTP Only Cookies** لحماية بيانات المستخدمين وتأمين الوصول إلى النظام.

1. يقوم المستخدم بإدخال بيانات تسجيل الدخول.
2. يتم التحقق من صحة البيانات المدخلة.
3. يتم إنشاء رمز مصادقة (JWT) للمستخدم.
4. يتم حفظ الرمز داخل HTTP Only Cookie.
5. يتم التحقق من الرمز في كل طلب من خلال Middleware مخصص قبل السماح بالوصول إلى الموارد المحمية.

كما تم تطبيق نظام التحكم في الصلاحيات المعتمد على الأدوار (RBAC) لإدارة صلاحيات المستخدمين المختلفة، مثل المستخدم العادي والفني، بحيث يتم تحديد مستوى الوصول وفقاً لدور كل مستخدم داخل النظام.

## واجهات البرمجة (REST API)

يوفر الخادم مجموعة من واجهات البرمجة (APIs) التي تسمح بالتفاعل مع النظام وإدارة البيانات.

### APIs الخاصة بالمصادقة:

- إنشاء حساب جديد (Register User)
- تسجيل الدخول (Login User)
- تسجيل الخروج (Logout User)
- جلب بيانات المستخدم الحالي (Get Current User)

### APIs الخاصة بطلبات الصيانة:

- إنشاء طلب صيانة جديد (Create Order)
- عرض طلبات المستخدم (View User Orders)

## خصائص الأمان (Security Features)

تم تضمين مجموعة من الإجراءات الأمنية لحماية النظام وبيانات المستخدمين، منها:

- تشفير كلمات المرور باستخدام bcryptjs.
- استخدام JWT للتحقق من هوية المستخدم.
- تخزين رمز المصادقة داخل HTTP Only Cookies.
- التحقق من صحة البيانات باستخدام Joi Validation.
- حماية المسارات الحساسة داخل النظام.
- تطبيق نظام إدارة الصلاحيات (Role-Based Authorization).

## مميزات هيكلية الخادم

توفر هذه الهيكلية العديد من المميزات المهمة، ومنها:

- توفير نظام مصادقة وصلاحيات آمن وموثوق.
- تنظيم الكود بطريقة تسهل التطوير والصيانة.
- تحسين عملية التواصل مع قاعدة البيانات.
- سهولة التكامل مع الواجهة الأمامية.

- دعم التوسع وإضافة مزايا جديدة مستقبلاً.

وتوفر هذه الهيكلية أساساً قوياً لإدارة طلبات الصيانة داخل المنصة، كما تسمح بإضافة وظائف مستقبلية مثل إدارة الفنيين، وتتبع حالة الخدمات، والإشعارات، ولوحات التحكم المتقدمة.

## 7. تصميم قاعدة البيانات (Database Design)

تعتمد منصة صيانتني (Sa2yanti) على MongoDB Atlas كقاعدة بيانات رئيسية لتخزين وإدارة بيانات المستخدمين وطلبات الصيانة. وتم تصميم قاعدة البيانات بحيث تكون مرنة وقابلة للتوسع، مما يساعد على إدارة البيانات بكفاءة مع إمكانية إضافة مزايا جديدة مستقبلاً.

### التقنيات المستخدمة (Technologies Used)

تم استخدام التقنيات التالية في طبقة قاعدة البيانات:

- MongoDB Atlas
- Mongoose ODM

تعتمد MongoDB على تخزين البيانات في صورة **Collections** و **Documents**، بينما يُستخدم Mongoose لإنشاء المخططات (Schemas) وإدارة العمليات المختلفة على قاعدة البيانات بطريقة منظمة.

### مجموعة المستخدمين (User Collection)

تُستخدم مجموعة **User** لتخزين بيانات المستخدمين المسجلين داخل المنصة.

أهم البيانات المخزنة:

- الاسم (Name)
- البريد الإلكتروني (Email)
- رقم الهاتف (Phone)
- كلمة المرور (Password)
- الدور (Role)

أنواع المستخدمين المدعومة:

- User (صاحب سيارة)
- Technician (فني صيانة)

يساعد نظام الأدوار في تحديد الصلاحيات الممنوحة لكل مستخدم داخل النظام.

### مجموعة طلبات الصيانة (Order Collection)

تُستخدم مجموعة **Order** لتخزين طلبات الصيانة التي ينشئها المستخدمون.

## أهم البيانات المخزنة:

- معرف المستخدم (User ID)
- معرف الفني (Technician ID)
- نوع الخدمة (Service)
- الموقع (Location)
- حالة الطلب (Status)

## حالات الطلب المتاحة:

- Pending ( قيد الانتظار )
- Accepted ( تم القبول )
- Completed ( مكتمل )

يساعد هذا النظام في متابعة حالة الطلب منذ إنشائه وحتى الانتهاء من تنفيذه.

## العلاقات بين البيانات (Relationships)

تعتمد المنصة على مجموعة من العلاقات التي تضمن تنظيم البيانات وإدارتها بشكل صحيح، وهي:

- يمكن للمستخدم الواحد إنشاء عدة طلبات صيانة.
- يمكن للفني الواحد إدارة عدة طلبات صيانة.

وتساعد هذه العلاقات على تتبع الطلبات وربطها بالمستخدمين والفنيين بسهولة.

## التحقق من البيانات والأمان (Data Validation and Security)

لضمان صحة البيانات وحماية النظام، تم تطبيق مجموعة من الإجراءات الأمنية، منها:

- استخدام Joi للتحقق من صحة البيانات قبل حفظها في قاعدة البيانات.
- تشفير كلمات المرور باستخدام bcryptjs.
- حماية العمليات الحساسة من خلال Middleware خاص بالمصادقة والصلاحيات.
- التحقق من أدوار المستخدمين قبل السماح بالوصول إلى الموارد المحمية.

## مميزات تصميم قاعدة البيانات

يوفر تصميم قاعدة البيانات العديد من المميزات، أهمها:

- هيكل مرن وقابل للتوسع.
- سرعة وكفاءة في تخزين واسترجاع البيانات.
- سهولة التكامل مع تطبيقات Node.js.
- دعم إضافة مزايا ووحدات جديدة مستقبلاً.
- تنظيم البيانات بطريقة تسهل إدارتها وصيانتها.

ويُعد هذا التصميم أساساً قوياً لإدارة بيانات المستخدمين والفنيين وطلبات الصيانة داخل منصة صيانتني، كما يدعم التوسع المستقبلي للمنصة وإضافة المزيد من الخدمات والخصائص.

## 8. التنفيذ والاعتماديات (Implementation & Dependencies)

تم تطوير منصة صيانتني (Sa2yanti) باستخدام بنية Full Stack تجمع بين الواجهة الأمامية (Frontend) ، والخادم (Backend) ، وقاعدة البيانات (Database) ، بهدف توفير نظام متكامل لإدارة طلبات صيانة السيارات بشكل آمن ومنظم.

### الوظائف المنفذة (Implemented Features)

يتضمن الإصدار الحالي من المنصة مجموعة من الوظائف الأساسية التي تم تنفيذها بنجاح، وتشمل:

- إنشاء حسابات جديدة للمستخدمين.
- تسجيل الدخول وتسجيل الخروج.
- نظام مصادقة باستخدام JWT و HTTP Only Cookies.
- إدارة الصلاحيات وفقاً لأدوار المستخدمين. (Role-Based Authorization)
- إنشاء طلبات صيانة جديدة.
- حماية الصفحات والمسارات التي تتطلب تسجيل الدخول.
- التحقق من صحة البيانات المدخلة.
- الربط والتعامل مع قاعدة البيانات.

### التقنيات المستخدمة (Technologies Used)

#### الواجهة الأمامية (Frontend)

تم استخدام التقنيات التالية لتطوير واجهة المستخدم:

- React.js
- TypeScript
- Tailwind CSS
- React Router DOM
- Axios
- React Hook Form
- Zod

#### الخادم (Backend)

تم استخدام التقنيات التالية لتطوير الخادم وإدارة العمليات الخلفية:

- Node.js
- Express.js
- MongoDB Atlas
- Mongoose
- JWT
- bcryptjs
- Joi
- cookie-parser

يوفر الإصدار الحالي من المنصة العديد من المميزات المهمة، منها:

- نظام مصادقة آمن لحماية حسابات المستخدمين.
- واجهة استخدام متجاوبة وسهلة الاستخدام.
- بنية تقنية قابلة للتوسع والتطوير مستقبلاً.
- تنظيم الكود بطريقة تسهل الصيانة وإضافة المزايا الجديدة.
- تكامل فعال بين الواجهة الأمامية والخادم وقاعدة البيانات.

ويُعد هذا الإصدار خطوة أساسية في بناء المنصة، حيث يوفر الوظائف الرئيسية المطلوبة لإدارة طلبات الصيانة، كما يشكل قاعدة قوية يمكن الاعتماد عليها لإضافة مزايا متقدمة مستقبلاً مثل إدارة الفنيين، وتتبع الطلبات بشكل مباشر، والإشعارات، والخدمات المعتمدة على الموقع الجغرافي.

## 9. الإنجازات والتقدم المحرز (Achievements & Progress)

تمكنت منصة صيانتني (Sa2yanti) من إكمال المرحلة الأساسية من التطوير بنجاح، حيث تم تنفيذ وربط مجموعة من الوظائف الرئيسية التي تمثل الأساس الذي تعتمد عليه المنصة في إدارة طلبات الصيانة والتعامل مع المستخدمين.

### الوظائف التي تم تنفيذها (Completed Features)

تم الانتهاء من تطوير وتنفيذ المزايا التالية:

- إنشاء حسابات المستخدمين وتسجيل الدخول.
- نظام مصادقة آمن باستخدام JWT و HTTP Only Cookies.
- إدارة الصلاحيات بناءً على دور المستخدم. (User & Technician)
- إنشاء نماذج قاعدة البيانات الخاصة بالمستخدمين وطلبات الصيانة.
- تطوير واجهة برمجية (API) لإنشاء طلبات الصيانة.
- حماية الصفحات والمسارات التي تتطلب تسجيل الدخول.
- التحقق من صحة البيانات باستخدام Joi و Zod.
- تطوير الصفحة الرئيسية. (Landing Page)
- تطوير صفحة تسجيل الدخول. (Login Page)
- تطوير صفحة إنشاء حساب جديد. (Register Page)
- الربط بين الواجهة الأمامية والخادم وقاعدة البيانات.

### الاختبارات (Testing)

تم اختبار الوظائف المنفذة للتأكد من عملها بشكل صحيح باستخدام أدوات مختلفة مثل Postman واختبارات المتصفح، وشملت الاختبارات:

- اختبار تسجيل الدخول وإنشاء الحسابات.
- التحقق من إدارة الصلاحيات للمستخدمين.
- اختبار إنشاء طلبات الصيانة.
- اختبار الوصول إلى الصفحات والمسارات المحمية.
- التحقق من التواصل بين الواجهة الأمامية والخادم.

## الحالة الحالية للمشروع (Current Status)

تم الانتهاء منه (Completed)

- نظام المصادقة. (Authentication System)
- نظام إدارة الصلاحيات. (Authorization System)
- الربط مع قاعدة البيانات.
- تطوير الجزء الخلفي الخاص بطلبات الصيانة.
- ربط واجهات البرمجة. (API Integration)
- تطوير صفحات المصادقة في الواجهة الأمامية.

قيد التطوير (In Progress)

- واجهة إدارة طلبات الصيانة.
- صفحة "طلباتي". (My Orders)
- لوحة تحكم المستخدم. (User Dashboard)

## الأعمال المستقبلية (Future Work)

يخطط المشروع لإضافة مجموعة من المزايا الجديدة خلال المراحل القادمة، ومنها:

- لوحة تحكم خاصة بالفنيين.
- إدارة وتحديث حالة الطلبات.
- نظام إشعارات فوري.
- دمج الخرائط والخدمات المعتمدة على الموقع الجغرافي.
- نظام التقييمات والمراجعات.

## ملخص التقدم

حقق المشروع تقدماً جيداً خلال مرحلة التطوير الحالية، حيث تم الانتهاء من بناء الوظائف الأساسية التي يعتمد عليها النظام، وأصبح يمتلك بنية مستقرة وأمنة يمكن البناء عليها مستقبلاً. كما يسير العمل وفق الخطة الموضوعية لإضافة المزايا المتبقية وتحويل المنصة إلى نظام متكامل لإدارة خدمات صيانة السيارات.

## 10. التحديات والأعمال المستقبلية (Challenges & Future Work)

خلال تطوير منصة صيانتى (Sa2yanti) واجه المشروع عددًا من التحديات التقنية التي تطلبت دراسة وحلولًا مناسبة لضمان بناء نظام آمن ومستقر.

من أبرز التحديات التي واجهت المشروع تنفيذ نظام المصادقة (Authentication) بطريقة آمنة. ففي البداية كان الاعتماد على تخزين بيانات المصادقة داخل **Local Storage**، ولكن تم لاحقًا الانتقال إلى استخدام **JWT مع HTTP Only Cookies** لتحسين مستوى الأمان وحماية بيانات المستخدمين. وقد تطلب هذا التغيير إجراء تعديلات على كل من الواجهة الأمامية والخادم لضمان عمل النظام بشكل صحيح.

كما واجه المشروع تحديًا آخر يتمثل في تطبيق نظام التحكم في الصلاحيات المعتمد على الأدوار (**Role-Based Access Control - RBAC**)، وذلك لضمان وصول كل مستخدم إلى الموارد المسموح له بها فقط وفقًا لدوره داخل النظام. وتمت معالجة هذا التحدي من خلال تطوير **Middleware** خاص بالمصادقة والصلاحيات، بالإضافة إلى تصميم واجهات برمجية آمنة للتحكم في الوصول إلى البيانات والعمليات المختلفة.

ومن التحديات الأخرى التي تم التعامل معها عملية الربط بين الواجهة الأمامية والخادم، والتأكد من تبادل البيانات بشكل صحيح وأمن بين مختلف أجزاء النظام.

### الأعمال المستقبلية (Future Work)

يوفر الإصدار الحالي الوظائف الأساسية للمنصة، إلا أن هناك العديد من المزايا المخطط إضافتها خلال المراحل القادمة من التطوير، ومنها:

- تطوير صفحة "طلباتي" (My Orders)
- إنشاء لوحة تحكم خاصة بالفنيين (Technician Dashboard)
- إدارة وتحديث حالات الطلبات بشكل متكامل.
- تطوير نظام إشعارات فوري للمستخدمين والفنيين.
- إضافة خدمات تعتمد على الموقع الجغرافي.
- إنشاء نظام للتقييمات والمراجعات.
- توفير تقارير وإحصائيات متقدمة لدعم اتخاذ القرار.

ستساهم هذه المزايا في تحسين تجربة المستخدم، وزيادة كفاءة إدارة الخدمات، وتحويل المنصة إلى نظام متكامل لإدارة صيانة المركبات.

### الخلاصة (Summary)

نجح مشروع صيانتى في بناء أساس قوي وآمن وقابل للتوسع لإدارة طلبات صيانة السيارات من خلال منصة إلكترونية حديثة تعتمد على تقنيات **Full Stack**. وقد تم تنفيذ الوظائف الأساسية المطلوبة وربط مكونات النظام المختلفة بنجاح، مما يوفر بيئة مستقرة يمكن تطويرها وإضافة مزايا جديدة إليها مستقبلاً.

وستركز مراحل التطوير القادمة على تحسين تجربة المستخدم، وإضافة خدمات متقدمة، وتوسيع إمكانيات المنصة بما يلبي احتياجات أصحاب السيارات ومقدمي خدمات الصيانة بشكل أكثر كفاءة وفاعلية.